

Graph Structure Learning for Traffic Prediction

Mahmood Amintoosi^{1*}

^{1*}Division of Computer Science, Department of Applied Mathematics,
Faculty of Mathematical Sciences, Ferdowsi University of Mashhad,
Bahonar Av., Mashhad, 91775, Khorasan-e Razavi, IRAN,
ORCID:0000-0001-9640-6475.

Corresponding author(s). E-mail(s): m.amintoosi@um.ac.ir;

Abstract

Accurate traffic forecasting remains challenging due to complex spatial and temporal dependencies. Existing graph-based methods typically rely on predefined adjacency matrices constructed from physical proximity, although physical connections may not adequately represent functional dependencies in traffic data. To address this limitation, we investigate a Graph Structure Learning (GSL) framework in which the adjacency matrix is learned directly from traffic observations rather than predefined from physical connections. We consider both contemporaneous and multi-lag learned graph structures and evaluate them with spatial and spatiotemporal forecasting models on the real-world SZ-Taxi and Los-loop datasets. The results show that learned graph structures can improve traffic forecasting, with the largest and most consistent gains obtained when lag-specific structures are consumed according to their temporal alignment. On Los-loop, the best learned multi-lag configuration reduces RMSE by 14.5% baseline at the first forecasting horizon and by up to 43.0% the physical-graph baseline across the evaluated horizons. On SZ-Taxi, the learned multi-lag structure provides only limited gains, reflecting the much sparser structure recovered from the data. Additional experiments with matched-sparsity graphs show that the observed gains on Los-loop cannot be explained by sparsity alone. Overall, the results indicate that learning the graph structure from traffic data can provide useful predictive information, although its effectiveness depends on the dataset and on how the learned structure is incorporated into the forecasting model. The learned graphs are interpreted as statistical dependencies rather than causal relationships. The paper code is available at <https://github.com/mamintoosi/TGCN-GSL-PyTorch>.

Keywords: Traffic forecasting, Graph structure learning, Graph convolutional networks, T-GCN, Multi-lag graphs, Graph consumption

1 Introduction

Accurate traffic forecasting is a core problem in intelligent transportation systems: navigation, ride-hailing, route planning, and congestion management all depend on reliable short-horizon speed or flow estimates [1]. The task is difficult because traffic state is both *spatial* — downstream roads respond to upstream conditions, and commuting couples distant corridors — and *temporal* — patterns evolve over minutes and hours. Graph-based deep models, especially Graph Convolutional Networks (GCNs) [2] and the temporal extension T-GCN [3], have become a standard tool: sensors are nodes, an adjacency matrix $\mathbf{A}$ encodes spatial coupling, and a recurrent or convolutional backbone maps a window of past speeds to future speeds.

In almost all of these systems, $\mathbf{A}$ is *given*. It is built from road connectivity, sensor distance, or a heuristic kernel. That default encodes a strong assumption: physical proximity approximates functional traffic dependency. The assumption is often false. Distant roads can be strongly coupled by commuter flows; adjacent sensors can be nearly independent. A second, less discussed failure mode is *oversmoothing*: when $\mathbf{A}$ is dense, repeated graph convolution averages away node-specific signal, so a model with a rich graph can be worse than the same model with *no* graph.

Graph structure learning (GSL) offers a data-driven alternative: estimate $\mathbf{A}$ from the traffic series rather than from the map [4]. Continuous score-based DAG learners such as NOTEARS [5] and DAGMA [6] make this practical by replacing combinatorial search with smooth optimization. Applying such learners to traffic is attractive: the resulting sparse adjacency can be read as a statistical dependency graph over sensors.

Yet two design questions remain open, and they organize this paper.

1. **What is the right reference graph?** If a dense physical graph is harmful at the operating point of interest, then a learned graph that merely *improves on the physical baseline* may still be worse than using the identity (graph-free) adjacency. Without a graph-free control, GSL gains are not identified.
2. **How is the graph consumed over time?** A learned adjacency can be applied as one static operator to the whole input window, or as lag-specific operators aligned with input timesteps. Recurrent backbones can use different operators at different steps; non-recurrent models cannot. *Structure* and *consumption* are separate design dimensions.

A system bibliography of the field makes the first question concrete: a survey of 784 traffic-forecasting articles (2000–2025) shows that graph-based methods have become dominant while still relying almost universally on heuristic graph construction (Appendix C). The literature rarely reports a graph-free control, so it is hard to tell whether reported GSL gains come from learned structure or from removing a harmful default.

This study

We evaluate GCN and T-GCN backbones on two public benchmarks — Los-loop (207 highway sensors, 5 min sampling) and SZ-Taxi (156 urban sensors, 15 min sampling) —

under one protocol that crosses four *graph sources* with four *consumption mechanisms*: physical road-network graphs; identity (graph-free) controls; learned *contemporaneous* DAGMA graphs (GSL and its binary symmetrization cGSL); and learned *multi-lag* DAGMA graphs consumed by a fixed cyclic assignment, a global lag mixture, a per-node per-timestep gate (T-GCN-MultiGSL-Mix), or a static union inside GCN. We use four horizons, five training seeds, and matched-sparsity and capacity controls.

Proposed method (summary).

The technical core is the separation of *estimating* lag-specific dependency graphs from *using* them (full development in Sect. 3). We stack an explicit lag window $Z = [x(t-L), \dots, x(t)]$ and fit DAGMA once on that matrix, then extract binary lag-block adjacencies. Those static graphs are supplied to T-GCN variants that either cycle them across input steps (T-GCN-MultiGSL), mix them with three global scalars (Weighted), or reweight them per node and per timestep with a small gate (Mix). A GCN counterpart consumes the element-wise union of the same graphs as a single static operator, providing a controlled comparison of consumption on nearly identical edge sets.

Main findings.

(i) The physical graph is not a sound default: graph-free models beat it in every dataset–horizon cell, on both backbones. (ii) Single learned contemporaneous graphs improve on the physical graph but never on the graph-free control. (iii) Multi-lag graphs help on Los-loop only when consumed in a temporally aligned way; the identical graphs unioned into one static adjacency fail. (iv) Gated mixing is the strongest Los-loop configuration (up to 14.5% relative RMSE reduction versus the graph-free baseline at PH1; 5/5 paired seeds; up to 43.0% versus the physical T-GCN baseline across horizons). (v) On SZ-Taxi, where the multi-lag fit retains only two edges, all learned variants stay within seed variability of graph-free; matched 30-edge controls show sparsity alone does not explain the Los-loop gain. Throughout, learned graphs are treated as *static statistical dependency estimates*, not causal maps.

Contributions.

1. A graph-free-baseline reassessment of graph structure learning for traffic forecasting on two public benchmarks.
2. A controlled dissociation indicating that graph consumption is a major determinant of whether multi-lag structure is useful (with the backbone confound stated explicitly).
3. A conditional positive result for gated per-timestep consumption of lag-specific learned graphs (T-GCN-MultiGSL-Mix).
4. Matched-budget evidence that learned edge placement, not sparsity alone, drives the remaining Los-loop gain.

Organization.

Section 2 reviews background, including the DAGMA estimator. Section 3 formalizes the task, both graph-learning constructions, and the consumption mechanisms.

Section 4 states the protocol. Section 5 reports results. Section 6 discusses interpretation. Section 7 lists limitations. Section 8 concludes.

2 Background and Related Work

2.1 Traffic forecasting: a brief landscape

Classical approaches model each series or a small neighborhood with ARIMA, SVR, Kalman filters, or shallow neural nets; they capture temporal regularity but struggle with network-scale spatial coupling [1, 7–9]. Deep spatio-temporal models (ST-ResNet, recurrent CNNs, LSTMs) improve temporal capacity but remain Euclidean unless a graph is introduced explicitly [10, 11].

Graph-based methods dominate recent traffic forecasting. GCN-style aggregation [2] and T-GCN [3] are the canonical lightweight backbones. Attention and transformer variants (A3T-GCN, dynamic spatial-temporal attention, graph transformers) reweight neighbors over time but still require an initial graph [12, 13]. In that sense they change *how* a given structure is used, not *which* structure is the right one. This paper deliberately isolates the structure/consumption question on GCN and T-GCN so that architecture scale does not confound the comparison.

2.2 Graph convolutional networks

A GCN layer maps node features $\mathbf{H}^{(l)}$ to $\mathbf{H}^{(l+1)}$ by

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right), \quad (2.1)$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ adds self-loops, $\tilde{\mathbf{D}}$ is the degree matrix of $\tilde{\mathbf{A}}$, and σ is a nonlinearity [2]. The normalized operator $\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$ aggregates neighborhood information. In our implementation every adjacency — physical, identity, and learned — is passed through the same Laplacian-with-self-loops normalization (Sect. 3.1), so the graph-free control is the identity operator in that pipeline, not a missing-graph special case.

2.3 Temporal graph convolutional networks

T-GCN [3] embeds graph convolution inside a Gated Recurrent Unit so that spatial aggregation and temporal gating interact at every input step. Writing $\mathbf{Z}_t$ for the graph-convolved features at time t , the GRU updates a hidden state $\mathbf{S}_t$ with update and reset gates; forecasting is trained with an MSE-type loss plus an ℓ_2 penalty on weights. T-GCN is the recurrent backbone for all multi-lag consumption mechanisms in this paper. The non-recurrent GCN backbone applies graph convolution to the flattened input window and produces the prediction in a single step — which is why it can only consume a *static* adjacency (Sect. 3.4).

2.4 Graph structure learning and DAGMA

Graph structure learning estimates $\mathbf{A}$ (or a weighted W) from data. In traffic, the motivation is that proximity is a poor proxy for functional co-movement [4]. Among GSL methods we use **DAGMA** [6], a continuous learner for directed acyclic graphs that extends the NOTEARS program [5].

From discrete DAGs to continuous optimization.

Learning a DAG over d variables by search is combinatorial. NOTEARS replaces the discrete constraint with a smooth equality $h(W) = 0$ on a real matrix $W \in \mathbb{R}^{d \times d}$, so that gradient-based methods apply. DAGMA strengthens this idea with a *log-determinant* acyclicity characterization based on M -matrices. For a sufficiently large scalar $s > 0$, define

$$h_{\text{ldet}}(W) = -\log \det(s\mathbf{I} - W \circ W) + d \log s, \quad (2.2)$$

where $\circ$ is the Hadamard product. Then $h_{\text{ldet}}(W) = 0$ if and only if the graph induced by W is acyclic; positive values quantify violation. Because h_{ldet} is smooth where defined, DAGMA can optimize a data-fit score under (or penalized by) this constraint via an augmented Lagrangian,

$$\mathcal{L}^\rho(W, \alpha) = \text{score}(W) + \frac{\rho}{2} |h_{\text{ldet}}(W)|^2 + \alpha h_{\text{ldet}}(W), \quad (2.3)$$

with dual variable α and penalty $\rho > 0$.

Linear SEM score used here.

We use the linear structural-equation variant (**DagmaLinear**) with an ℓ_2 loss and an ℓ_1 sparsity penalty λ_1 on the coefficients. After optimization, a binary adjacency is obtained by an absolute-magnitude threshold on $|W_{ij}|$ (sign-symmetric, matching the library semantics). In our code path the threshold regimes differ by construction — an internal fit threshold for the contemporaneous graphs and a consumer threshold for the multi-lag graphs — and are specified exactly in Sects. 3.2 and 3.3.

Interpretation limits.

Two cautions matter for the rest of the paper. First, acyclicity is an *optimization regularizer* that makes continuous learning tractable; it is not evidence that traffic influence is acyclic, and we never claim causal discovery. Second, the learned W is a *statistical* dependency estimate under linearity and noise assumptions. Where we speak of lag blocks, that reading is tied to how Z is built (Sect. 3.3), not to a validated physical propagation model.

2.5 Structure versus consumption

Much of the GSL-for-traffic literature treats a learned adjacency as a drop-in replacement for a physical one: same backbone, new $\mathbf{A}$. That framing silently assumes

consumption is irrelevant. Recurrent models, however, can apply different graph operators at different input steps, and non-recurrent models cannot. This paper makes the distinction explicit: three lag-specific DAGMA graphs are shared by T-GCN variants (fixed cyclic assignment, global weights, or a per-node gate) and by a GCN that consumes their static union. Empirically, that shared-graph design is what allows us to ask whether utility attaches to the edge set or to the way it is used (Sects. 5.4 and 6.2).

2.6 Evaluation practice

Reported GSL gains are usually relative to a physical or heuristic graph. A graph-free identity control, matched-sparsity controls, and multi-seed reporting are less common, but they are exactly what is needed if learned structure is to be credited with gains that *removing* a harmful graph already provides. This study therefore treats NoSpatial as a first-class baseline rather than an ablation curiosity.

3 Problem Formulation and Graph Learning Framework

This section defines the forecasting task, the graph constructions, and the mechanisms by which the forecasting backbones consume graph structure. Two design dimensions are kept strictly separate throughout: the *source* of the graph (physical, identity, learned contemporaneous, or learned multi-lag) and the *consumption* of the graph (one static operator versus timestep-specific operators). Throughout, $\mathbf{A}$ denotes the binary adjacency consumed by the forecasting backbone, whereas W denotes the continuous coefficient matrix estimated by the graph-structure-learning procedure; the two symbols are not interchangeable.

3.1 Forecasting Task and Model Baselines

Let $x(t) \in \mathbb{R}^N$ denote the traffic speeds observed at N sensors at time step t , where one step corresponds to the sampling interval of the dataset (5 min for Los-loop, 15 min for SZ-Taxi; Sect. 4). Given an input window of length

$$T = 12,$$

the forecasting task is to map

$$\underbrace{[x(t-11), \dots, x(t)]}_{X \in \mathbb{R}^{T \times N}} \longmapsto \underbrace{[x(t+1), \dots, x(t+PH)]}_{Y \in \mathbb{R}^{PH \times N}},$$

where the prediction horizon $PH \in \{1, 2, 3, 4\}$ is expressed in steps of the sampling interval. The PH -dimensional target at every sensor is produced jointly in a single forward pass by a shared linear output layer; prediction is multi-step but *not* autoregressive, and no predicted value is fed back into the model.

The backbone adjacency is a binary matrix

$$\mathbf{A} \in \{0, 1\}^{N \times N}.$$

It is produced from a continuous coefficient matrix

$$W \in \mathbb{R}^{d \times d}, \quad d = \begin{cases} N & (\text{contemporaneous construction, Sect. 3.2}), \\ (L + 1)N & (\text{multi-lag construction, Sect. 3.3}). \end{cases}$$

$\mathbf{A}$ denotes the backbone adjacency, whereas W denotes the continuous DAGMA coefficient matrix.

Two reference configurations fix the ends of the graph-source spectrum:

- **Graph-free control.** $\mathbf{A} = \mathbf{I}_N$, the identity adjacency, named *T-GCN-NoSpatial* and *GCN-NoSpatial* in the two backbone families. The identity adjacency is passed through exactly the same graph-normalization pipeline as every other graph configuration (see “Graph normalization” below), so this control isolates the contribution of spatial structure rather than of the graph operator itself.
- **Physical baseline.** The dataset-provided road-network adjacency, named *T-GCN* in the T-GCN family and *GCN* in the GCN family.

The two backbones are deliberately lightweight. GCN applies graph convolution to the flattened input representation and produces the hidden state in a single step; T-GCN incorporates the same graph convolution into a GRU-based recurrent architecture that updates a hidden state at every input step. Generic derivations of both architectures are given in Sect. 2; here we only note the properties that matter for the experimental design: all graph configurations are consumed by architecturally identical backbones, differing only in the adjacency supplied.

Graph normalization.

Every adjacency — physical, identity, and all learned graphs — is transformed by the same operator, following the GCN convention of Kipf and Welling [2]. With $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ and $\tilde{\mathbf{D}}_{ii} = \sum_j \tilde{\mathbf{A}}_{ij}$, the operator is

$$\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}}^\top \tilde{\mathbf{D}}^{-1/2}. \quad (3.1)$$

Self-loops are therefore added before normalization, the normalization is symmetric, and the transposed form in Eq. (3.1) is the one actually implemented. Because $\tilde{\mathbf{A}}$ is symmetric whenever $\mathbf{A}$ is (as holds for the physical adjacency after symmetrization-free loading of its symmetric datasets, the identity, and the cGSL graphs), the operator coincides with the standard symmetrically-normalized adjacency in those cases; for the directed learned graphs it realizes the same row/column scaling on the transposed matrix. For the identity adjacency the operator reduces exactly to $\hat{\mathbf{A}} = \mathbf{I}$, confirming that the graph-free control differs from the other configurations only in the absence of off-diagonal structure.

3.2 Contemporaneous Graph Learning (GSL and cGSL)

The first learned construction estimates a single dependency graph among the N sensors from *simultaneous* observations. We use DAGMA [6], which estimates a weighted adjacency $W \in \mathbb{R}^{N \times N}$ by minimizing a linear structural-equation-model score — an ℓ_2 loss on the reconstructed observations — augmented with an ℓ_1 sparsity penalty $\lambda_1 \|W\|_1$, subject to the DAGMA acyclicity characterization

$$h(W) = -\log \det(d\mathbf{I} - W \circ W) + d \geq 0,$$

which holds if and only if $W \circ W$ is an M-matrix, i.e., W is acyclic; h is enforced numerically during the constrained optimization.

Exact construction.

The DAGMA input is the subsampled matrix of training-split snapshots

$$\mathbf{X} = \text{train_norm}[0 :: \text{PH}],$$

that is, every PH-th training row, so each row is one simultaneous observation of all N sensors and columns correspond to sensors. There are N variables and *no* lag blocks. For Los-loop the inputs contain 1612, 806, 538, and 403 rows for $\text{PH} = 1, \dots, 4$; for SZ-Taxi the corresponding counts are 2380, 1190, 794, and 595. The construction is therefore PH-dependent through its input subsampling, in contrast to the multi-lag construction of Sect. 3.3.

The sparsity penalty is $\lambda_1 = 0.02$ for Los-loop and $\lambda_1 = 0.01$ for SZ-Taxi, and DAGMA’s internal magnitude threshold is set to 0.3: after the fit, coefficients with $|W_{ij}| < 0.3$ are zeroed inside the estimation procedure. The binary support is then extracted with the absolute-magnitude rule

$$\mathbf{A}_{\text{GSL}} = \mathbf{1}(|W| > 0),$$

with the diagonal removed. Under this construction the support rule is equivalent to retaining exactly those coefficients whose magnitude reaches the internal threshold of 0.3. This 0.3-based regime is specific to the contemporaneous construction and must not be conflated with the 0.1 consumer threshold used by the multi-lag construction (Sect. 3.3). The graph is learned using training data only.

The resulting directed off-diagonal edge counts are 28 for Los-loop and 8 for SZ-Taxi, identical across $\text{PH} = 1, \dots, 4$.

cGSL.

The second learned configuration is not a separately learned graph: it is the symmetrized version of the very same GSL artifact,

$$\mathbf{A}_{\text{cGSL}} = \mathbf{1}(\mathbf{A}_{\text{GSL}} + \mathbf{A}_{\text{GSL}}^\top > 0),$$

with the diagonal removed. No second DAGMA fit and no second threshold are involved. The symmetrized graphs contain 56 (Los-loop) and 16 (SZ-Taxi) directed entries. The same graph artifacts are shared by the GCN and T-GCN counterparts, so both backbone families see identical adjacencies.

Interpretation restriction.

The contemporaneous DAGMA graph is a statistical dependency structure estimated from simultaneous observations. It is *not* a temporal graph: an entry $\mathbf{A}_{ij} = 1$ does not mean that sensor j at time t predicts sensor i at time $t + 1$. Acyclicity is an optimization and fitting constraint of the estimator and is not treated as evidence of a causal structure.

3.3 Multi-Lag Graph Learning

The second learned construction estimates dependency structure between lagged and current observations. With lag depth

$$L = 3,$$

the lag-stacked representation is

$$\mathbf{Z} = [x(t-3), x(t-2), x(t-1), x(t)],$$

a matrix with $(L+1)N$ columns: $4 \times 207 = 828$ variables for Los-loop and $4 \times 156 = 624$ for SZ-Taxi. The canonical construction contains 1609 training windows for Los-loop and 2377 for SZ-Taxi. DAGMA is fitted *once* on the training split, and the learned artifact is shared across $\text{PH} = 1, \dots, 4$:

The multi-lag graph is PH-independent: the same learned graph artifact is consumed for all four prediction horizons. Unlike the multi-lag construction, the contemporaneous graph is PH-dependent because its input is subsampled as `train_norm[0 :: PH]`.

Multi-lag optimization.

The ℓ_1 coefficient is $\lambda_1 = 0.01$ for both datasets. In contrast to the contemporaneous construction, DAGMA’s internal threshold is 0.0: raw continuous weights are retained, and thresholding is deferred to the consumer. The two threshold regimes are kept explicit throughout:

1. *Contemporaneous GSL*: internal DAGMA threshold 0.3; the support is extracted after that internal threshold.
2. *Multi-lag*: internal DAGMA threshold 0.0; raw weights retained; the consumer thresholds at $|W| > 0.1$, with the diagonal removed.

Block interpretation.

In the DAGMA convention W_{ij} quantifies the estimated dependency of variable j on variable i . The block $W[lN : (l+1)N, LN : (L+1)N]$ therefore collects, for each pair

of sensors, the estimated dependency of the current snapshot on the snapshot $L - l$ steps earlier; after the consumer threshold this yields the lag-specific adjacency $\mathbf{A}_k$, $k \in \{1, 2, 3\}$, read as “activity at $t - k$ associated with activity at t ”. The contemporaneous block ($x(t) \rightarrow x(t)$) is present in the DAGMA fit but is *not* consumed by any of the forecasting methods in this study.

Edge counts.

After the consumer threshold, the per-lag off-diagonal edge counts are:

- **Los-loop:** lag-1: 12; lag-2: 3; lag-3: 15; total lagged edge slots: 30; union: 28 distinct off-diagonal edges.
- **SZ-Taxi:** lag-1: 0; lag-2: 0; lag-3: 2; total: 2; union: 2.

Edge *slots* count entries per lag block and may repeat an edge across lags; the *union* counts distinct sensor pairs after merging the three blocks.

Interpretation restrictions.

The learned lag graphs are descriptive statistical dependency estimates. The lag-structured reading is consistent with their construction and with the consumption experiments below, but the graphs are not independently validated as causal or predictive structures, and DAGMA’s acyclicity constraint is not causal evidence. The graphs are static: they are fitted once on the training split and never change across the input window or across evaluation samples; only the manner in which they are *consumed* can vary with the input timestep, as described next.

3.4 Graph Consumption Mechanisms

Graph source and graph consumption are treated as separate design dimensions. A graph with a given edge set can be consumed in different ways, and the same consumption mechanism can be applied to different graphs. This subsection defines the four consumption mechanisms used in the study; the corresponding empirical comparison is presented in Sect. 5.

Static graph mechanisms.

For the physical baseline, the graph-free control, and the two contemporaneous learned graphs (GSL and cGSL), one adjacency is used for all input steps of the window. Graph-side extra parameters: 0.

Fixed cyclic lag assignment (T-GCN-MultiGSL).

The three lag-specific graphs $\mathbf{A}_1, \mathbf{A}_2, \mathbf{A}_3$ are consumed separately, one per input step. At input step t , where $t = 0$ denotes the most recent input step of the window, the graph index follows the rule

$$\text{graph_idx} = (T - 1 - t) \bmod 3, \quad (3.2)$$

so the most recent input step is convolved with the lag-1 graph, the preceding step with the lag-2 graph, and the oldest step of the window with the lag-3 graph; the assignment cycles for the remaining steps of the 12-step window. Extra parameters: 0.

Global lag mixture (T-GCN-MultiGSL-Weighted).

Three learned scalar weights, one per lag graph, are passed through a softmax, and the resulting convex combination of the three normalized lag-graph operators is used for every input step. Extra parameters: +3. This variant is treated as a supplementary mechanism.

Per-node, per-timestep gate (T-GCN-MultiGSL-Mix).

At each input step t and each node, a small gate network takes the pair $[x_t; h_{t-1}]$ — the current input and the recurrent hidden state — and produces a softmax over the $K = 3$ lag graphs. The gate output mixes the three normalized lag-graph operators into one operator for that node and step, which is then applied before the GRU-style update. Extra parameters: +4,419. Crucially, Mix varies *how* the static lag-specific graphs are consumed; it does not relearn or modify the graph edge set at any timestep, and it is not an adaptive-graph method: the learned structure itself never changes.

Static union in GCN (GCN-MultiGSL).

The GCN backbone consumes the whole input window in a single graph convolution, so per-timestep graph assignment is not applicable. GCN-MultiGSL therefore merges the three lag graphs into one static adjacency for the complete window,

$$\mathbf{A}_{\cup} = \bigvee_{k=1}^3 \mathbf{A}_k,$$

the element-wise maximum for binary adjacency matrices, normalized identically to all other graphs. The same multi-lag graph artifacts are thus consumed by GCN-MultiGSL through a static union and by T-GCN-MultiGSL through lag-specific, per-timestep consumption. The comparison is designed to assess whether the utility of the same learned multi-lag structure depends on how it is consumed; because GCN and T-GCN also differ in backbone architecture, the comparison does not isolate consumption as the sole differing factor.

Parameter counts.

All configurations within a backbone family share the same backbone capacity, and the graph source contributes no trainable parameters. Verified from the implementation, the trainable parameters (excluding the shared linear output layer) are 768 for GCN and 12,672 for T-GCN in every single-graph configuration; T-GCN-MultiGSL-Weighted adds 3 and T-GCN-MultiGSL-Mix adds 4,419 parameters relative to T-GCN-MultiGSL. The capacity implications of the Mix gate are examined separately in the experiments (Sect. 4).

4 Experimental Setup

This section describes the complete experimental protocol. All twelve forecasting configurations, the auxiliary controls, and the temporal-resolution variant share the same data handling, training procedure, and evaluation.

4.1 Datasets

Two widely used public traffic benchmarks are used [3]:

- **Los-loop.** Speed measurements from 207 highway loop detectors on Los Angeles County highways, collected in March 2012 and aggregated at 5-minute intervals; the series comprises 2016 timesteps (seven days).
- **SZ-Taxi.** Speed measurements from 156 urban road sensors in the Luohu District of Shenzhen, China, collected in January 2015 and aggregated at 15-minute intervals; the series comprises 2976 timesteps.

Each dataset provides a feature matrix (sensor speeds over time) and a road-network adjacency, which is used for the physical baseline (Sect. 3.1).

4.2 Data Split and Normalization

The series are split chronologically into 80% training and 20% test segments: 1612 training and 404 test timesteps for Los-loop, and 2380 training and 596 test timesteps for SZ-Taxi. Input windows and targets are generated separately within each segment, so no forecasting window crosses the train/test boundary.

Features are normalized by dividing by the *training-split* maximum only: 70.0 (km/h) for Los-loop and 86.4292 for SZ-Taxi. The test split is not used for normalization or graph learning.

4.3 Forecasting Protocol

The input window length is $T = 12$ and the prediction horizons are $\text{PH} \in \{1, 2, 3, 4\}$, expressed in steps of the dataset’s sampling interval and produced as the multi-step target defined in Sect. 3.1. In wall-clock terms the horizons correspond to:

- **Los-loop** (5-minute steps): $\text{PH1} = 5$ min, $\text{PH2} = 10$ min, $\text{PH3} = 15$ min, $\text{PH4} = 20$ min ahead.
- **SZ-Taxi** (15-minute steps): $\text{PH1} = 15$ min, $\text{PH2} = 30$ min, $\text{PH3} = 45$ min, $\text{PH4} = 60$ min ahead.

Because the two datasets use different sampling intervals, horizon indices are only comparable within, not across, datasets.

4.4 Methods

The study evaluates twelve configurations: seven in the T-GCN family and five in the GCN family, spanning the graph sources and consumption mechanisms defined in Sect. 3.

T-GCN family.

T-GCN (physical graph), T-GCN-NoSpatial (graph-free control), T-GCN-GSL, T-GCN-cGSL (learned contemporaneous graphs, static consumption), and three consumption mechanisms over the learned multi-lag graphs: T-GCN-MultiGSL (fixed cyclic lag assignment), T-GCN-MultiGSL-Weighted (global lag mixture), and T-GCN-MultiGSL-Mix (per-node, per-timestep gate).

GCN family.

GCN (physical graph), GCN-NoSpatial (graph-free control), GCN-GSL, GCN-cGSL (learned contemporaneous graphs), and GCN-MultiGSL (static union of the multi-lag graphs).

All configurations use architecturally identical backbones within each family and differ only in the adjacency supplied and, for the multi-lag T-GCN variants, in how the lag graphs are consumed (Sect. 3.4).

4.5 Training Protocol

All forecasting models are trained with Adam (learning rate 10^{-3} , weight decay 10^{-4}), batch size 128, for 50 epochs, with hidden dimension 64 and a shared linear output layer mapping the hidden state to the PH-dimensional target.

The training loss differs by backbone family, as in the reference implementations of the two architectures. The T-GCN family is trained with an ℓ_2 loss combined with an ℓ_2 penalty on the model weights,

$$\mathcal{L}_{\text{T-GCN}} = \frac{1}{2} \sum_i (\hat{y}_i - y_i)^2 + \lambda_{\text{reg}} \cdot \frac{1}{2} \sum_{\theta \in \Theta} \|\theta\|_2^2, \quad \lambda_{\text{reg}} = 1.5 \times 10^{-3},$$

where the sums run over prediction elements and over all trainable weights; the GCN family is trained with the plain mean squared error. Both families share the same output layer and optimizer settings.

Each configuration is trained under five seeds, $\{42, 43, 44, 45, 46\}$, and all reported statistics aggregate over these five training runs. The learned graphs are fitted once and are deterministic; they are shared across the five forecasting-model training seeds, so seed variability reflects the training and evaluation of the forecasting models only.

Graph learning uses DAGMA-linear [6] with the ℓ_2 loss type, 30,000 warm-up iterations and 60,000 maximum iterations (library defaults), and internal seed 42; the estimator is deterministic (zero initialization, no stochastic component) under this configuration. All graph-learning hyperparameters and threshold regimes are as specified in Sects. 3.2 and 3.3.

4.6 Evaluation Metrics

The primary metric is the root mean squared error (RMSE); the mean absolute error (MAE) is reported as a secondary metric. Both are computed on the de-normalized original speed scale. Test evaluation is performed on the complete test set in a single full-batch pass.

4.7 Statistical Reporting

Main-text numbers are five-seed means with sample standard deviations (ddof = 1). We report paired per-seed win counts and paired t -tests for reference. With $n = 5$, the exact Wilcoxon signed-rank test cannot fall below $p = 0.0625$; we therefore make no definitive significance claims.

4.8 Matched-Sparsity and Capacity Controls

Two supplementary controls probe alternative explanations of the multi-lag results. Both use the same training protocol as the main experiments, and both are scope-labeled: they are attached to a single experimental cell rather than to the complete configuration matrix, and their conclusions do not extend beyond that cell.

Matched sparsity (Los-loop, PH = 1).

On the Los-loop PH1 cell, the multi-lag construction consumes 30 lagged edge slots. This control compares, at the same 30-edge budget: RandTop30, a graph of 30 uniformly random directed edges, redrawn per seed; CorrTop30, the 30 largest-magnitude pairwise Pearson correlations of the training data; and the DAGMA multi-lag graphs under the fixed and the gated consumption mechanisms. Five seeds are used per graph. This control cell lies outside the complete 12-method dataset $\times$ horizon matrix and is reported separately. No sparsified variant of the physical graph and no sensitivity sweep over sparsity coefficients or thresholds was performed.

Capacity control.

The gated mechanism adds 4,419 parameters relative to T-GCN-MultiGSL (Sect. 3.4). To check that its results are not explained by capacity alone, a hidden-dimension-74 T-GCN-NoSpatial variant (16,872 parameters, matching the 17,091-parameter gated model more closely than the 12,672-parameter standard T-GCN) is compared against the hidden-64 NoSpatial and the gated model on the same cell. This check is single-seed and is labeled as such.

4.9 Temporal-Resolution Variant

To probe the effect of temporal resolution, the Los-loop series is aggregated by averaging consecutive groups of three 5-minute observations, yielding a 15-minute series of length $T = 672$ with an 80/20 split. Prediction horizons $\text{PH} \in \{1, 2, 3, 4\}$ correspond to 15, 30, 45, and 60 minutes ahead. This is a *temporal-resolution variant* of Los-loop, not a third dataset, and its PH indices must not be interpreted as

equivalent to PH indices in the original 5-minute Los-loop series. A separate PH-independent multi-lag DAGMA fit is performed on this training split ($\lambda_1 = 0.01$; consumer threshold $|W| > 0.1$), and three configurations are trained per horizon with five seeds under the standard protocol: T-GCN-NoSpatial, T-GCN-MultiGSL, and T-GCN-MultiGSL-Mix.

4.10 Implementation and Reproducibility

The implementation, data, and configuration files are available at <https://github.com/mamintoosi/TGCN-GSL-PyTorch>. Graph learning uses training data only and has no access to the test split, for both the contemporaneous and the multi-lag constructions and for all control graphs.

5 Results

This section reports the forecasting results under the protocol of Sect. 4. Unless stated otherwise, every value is the mean RMSE over the five training seeds $\{42, \dots, 46\}$ with the sample standard deviation in parentheses; graphs are fixed across seeds. Improvements are relative RMSE reductions with respect to the named reference. Statistical qualifications follow Sect. 4.7: with $n = 5$ we do not make definitive significance claims. Interpretation is deferred to the Discussion.

5.1 The Physical Graph Is Not a Sound Default

Table 1 reports the T-GCN family on both datasets. The physical road-network graph (T-GCN) is the worst configuration in its family in every dataset-horizon cell. On Los-loop at PH1, T-GCN attains 7.88 ± 0.28 RMSE, whereas the graph-free control T-GCN-NoSpatial attains 5.25 ± 0.19 — a relative RMSE reduction of 33.3% when the physical graph is removed. The same direction holds at PH2–PH4 on Los-loop (29.2%, 27.0%, and 24.1% relative to T-GCN) and at every horizon on SZ-Taxi (24.4%, 25.1%, 25.2%, 25.1%). Across all eight dataset $\times$ horizon cells, NoSpatial beats Physical in 5/5 seeds.

The GCN family (Table 2) shows the same reversal: GCN (physical) attains 8.14 ± 0.11 on Los-loop PH1 versus 4.88 ± 0.31 for GCN-NoSpatial, and remains worse than GCN-NoSpatial at every horizon on both datasets.

These results establish the graph-free control, not the physical graph, as the appropriate reference for evaluating learned structure at these operating points (structural contrast in Fig. 3).

5.2 A Single Learned Graph Does Not Close the Gap

The contemporaneous learned graphs GSL and cGSL (Sects. 3.2) improve on the physical graph in most T-GCN cells, but they do not close the gap to the graph-free control.

On Los-loop, T-GCN-GSL and T-GCN-cGSL obtain 5.86 ± 0.21 and 5.82 ± 0.23 at PH1, versus 5.25 ± 0.19 for NoSpatial: both are approximately 11% *worse* than removing the graph, while still reducing error by roughly 26% relative to the physical

baseline. The same ordering — Physical > single learned graph > NoSpatial in error — holds at PH2–PH4 on Los-loop and at every horizon on SZ-Taxi. In no dataset $\times$ horizon cell does either contemporaneous learned graph beat the graph-free control.

The GCN family is less uniform. GCN-cGSL (5.76 ± 0.20 on Los-loop PH1) beats the physical GCN but still loses to GCN-NoSpatial (4.88 ± 0.31). GCN-GSL is the notable exception: on Los-loop it is often comparable to, or worse than, the physical GCN (7.83 ± 0.11 at PH1) and does not consistently improve on that baseline. Symmetrization therefore helps the non-recurrent GCN backbone (cGSL < GSL in error on Los-loop PH1: 5.76 vs 7.83) while the directed and symmetrized variants are nearly indistinguishable for T-GCN (5.86 vs 5.82). Mechanism-level discussion of this asymmetry is deferred to Sect. 6.

In summary, replacing the physical adjacency with a single learned contemporaneous graph recovers only part of the error reduction obtained by removing the graph entirely. Motivated by this gap, the remainder of the results examine lag-specific multi-lag graphs and, critically, how those graphs are consumed.

5.3 Sparsity and Capacity Controls

Before interpreting any multi-lag gain, we report the matched-sparsity and capacity controls of Sect. 4.8. These controls exist **only for Los-loop at PH1** and do not establish conclusions for other cells.

At a matched budget of 30 directed edges, the random control RandTop30 obtains 6.10 ± 0.12 and the top-correlation control CorrTop30 obtains 5.39 ± 0.10 . Both are worse than the graph-free control on the same cell (5.25 ± 0.19). By contrast, the DAGMA multi-lag graphs at the same 30-edge budget obtain 4.84 ± 0.11 under fixed per-timestep consumption (T-GCN-MultiGSL) and 4.49 ± 0.14 under gated consumption (T-GCN-MultiGSL-Mix). Table 3 summarizes the comparison.

These matched-budget results show that sparsity alone does not explain the Los-loop PH1 multi-lag gain: two non-learned 30-edge graphs are worse than no graph, whereas DAGMA-placed edges with per-lag consumption are better. The conclusion is scoped to this single experimental cell.

The capacity control (single seed, seed 42, labeled as such) compares T-GCN-NoSpatial at hidden dimension 64 (12,672 parameters, RMSE 5.14) with a hidden-dimension-74 variant (16,872 parameters, RMSE 5.14) that more closely matches the parameter count of T-GCN-MultiGSL-Mix (17,091). Increasing capacity on the graph-free model changes RMSE by less than 0.02, whereas Mix reaches 4.49 at five seeds on the same cell. The Mix gain is therefore not explained by its additional 4,419 gate parameters alone.

5.4 Multi-Lag Graphs: Consumption Matters

We next compare configurations that use the *same* multi-lag DAGMA artifacts (Sect. 3.3) under different consumption mechanisms (Sect. 3.4).

On Los-loop at PH1, T-GCN-MultiGSL (fixed cyclic lag assignment; 0 extra parameters) obtains 4.84 ± 0.11 , improving on NoSpatial by 7.8%. T-GCN-MultiGSL-Weighted, which learns only three global scalars, obtains 4.83 ± 0.11 — effectively

identical to the fixed assignment. The global lag mixture does not add value beyond the fixed per-timestep rule.

The striking comparison is with GCN-MultiGSL, which consumes the *identical* three lag graphs as one static union $\mathbf{A}_\cup = \bigvee_k \mathbf{A}_k$ (28 of the 30 lagged edge slots appear in the union). GCN-MultiGSL obtains 9.78 ± 0.14 on Los-loop PH1: worse than the physical GCN (8.14 ± 0.11) and more than twice the error of T-GCN-MultiGSL on the same graphs (4.84 ± 0.11). The direction persists at PH2–PH4 (GCN-MultiGSL 10.05–10.27 versus T-GCN-MultiGSL 5.45–6.26).

Because the union holds nearly the same edges as the three lag graphs, the edge set alone cannot account for the difference between the two outcomes (Fig. 1). At the same time, GCN and T-GCN also differ in backbone architecture, so this comparison provides evidence that *how the multi-lag structure is consumed* is a major determinant of whether it is useful; it is not a fully controlled isolation of consumption as the sole differing factor. On SZ-Taxi, where the multi-lag graphs retain only 2 edges, the corresponding contrast is small in absolute RMSE.

5.5 Gated Mixing on Los-Loop: The Positive Result

T-GCN-MultiGSL-Mix, which gates the three static lag graphs per node and per input timestep, is the strongest configuration on Los-loop at every horizon (Table 1). Relative to the graph-free control, Mix reduces RMSE by 14.5%, 11.9%, 9.2%, and 10.9% at PH1–PH4. Relative to the physical T-GCN baseline, the corresponding reductions are 43.0%, 37.6%, 33.7%, and 32.3%.

At PH1, Mix attains 4.49 ± 0.14 versus 5.25 ± 0.19 for NoSpatial. The paired per-seed difference is positive in 5/5 seeds at every horizon (mean paired difference 0.76, 0.68, 0.56, and 0.72 RMSE at PH1–PH4; paired- t $p \leq 0.0015$ at PH1–PH2 and $p \leq 0.00002$ at PH4, reported for reference under the $n = 5$ caveat of Sect. 4.7). Mix also improves on the fixed MultiGSL assignment in 5/5 seeds at every horizon, by 0.35–0.40 RMSE on Los-loop (approximately 6–7% relative). Weighted remains within 0.01 RMSE of Fixed at every Los-loop horizon and is treated as a null ablation.

Figure 2 shows the per-seed distributions for NoSpatial, MultiGSL, Weighted, and Mix on Los-loop PH1–PH4; the means match Table 1.

This is the study’s main positive result: on Los-loop, lag-specific learned graphs improve on the graph-free baseline only when they are consumed in a temporally aligned manner, and the per-node gate adds a further consistent gain over the fixed assignment.

5.6 Boundaries: Temporal Resolution and Dataset Dependence

SZ-Taxi.

On SZ-Taxi, every learned variant remains within seed variability of the graph-free control. At PH1, T-GCN-MultiGSL-Mix obtains 4.119 ± 0.020 versus 4.120 ± 0.004 for NoSpatial (difference ≤ 0.015 RMSE at every horizon; Mix wins 4/5, 4/5, 5/5, and 4/5 paired seeds at PH1–PH4). Relative RMSE reductions versus NoSpatial are at most 0.34%. The learned multi-lag graphs on SZ-Taxi retain only 2 lagged edges

Table 1: T-GCN family: test RMSE (mean, sample standard deviation in parentheses) over five seeds. Physical: road-network graph. NoSpatial: identity adjacency. GSL/cGSL: learned contemporaneous graphs. MultiGSL family: learned multi-lag graphs under fixed, global-weighted, and gated consumption (Sect. 3).

Method	PH1	PH2	PH3	PH4
<i>Los-loop (5 min)</i>				
T-GCN (Physical)	7.88 (0.28)	8.13 (0.09)	8.37 (0.17)	8.66 (0.16)
T-GCN-NoSpatial	5.25 (0.19)	5.76 (0.08)	6.11 (0.07)	6.58 (0.12)
T-GCN-GSL	5.86 (0.21)	6.30 (0.06)	6.66 (0.12)	7.04 (0.09)
T-GCN-cGSL	5.82 (0.23)	6.35 (0.12)	6.67 (0.11)	7.09 (0.12)
T-GCN-MultiGSL	4.84 (0.11)	5.45 (0.14)	5.94 (0.03)	6.26 (0.03)
T-GCN-MultiGSL-Weighted	4.83 (0.11)	5.44 (0.13)	5.93 (0.03)	6.25 (0.03)
T-GCN-MultiGSL-Mix	4.49 (0.14)	5.08 (0.15)	5.55 (0.16)	5.86 (0.07)
<i>SZ-Taxi (15 min)</i>				
T-GCN (Physical)	5.45 (0.11)	5.55 (0.08)	5.60 (0.05)	5.63 (0.06)
T-GCN-NoSpatial	4.12 (0.00)	4.16 (0.01)	4.19 (0.01)	4.23 (0.01)
T-GCN-GSL	4.28 (0.05)	4.31 (0.03)	4.33 (0.01)	4.37 (0.02)
T-GCN-cGSL	4.30 (0.03)	4.34 (0.02)	4.38 (0.03)	4.41 (0.01)
T-GCN-MultiGSL	4.13 (0.02)	4.16 (0.00)	4.20 (0.01)	4.22 (0.00)
T-GCN-MultiGSL-Weighted	4.13 (0.02)	4.16 (0.00)	4.20 (0.01)	4.22 (0.00)
T-GCN-MultiGSL-Mix	4.12 (0.02)	4.15 (0.01)	4.18 (0.00)	4.22 (0.01)

(union 2), versus 30 lagged slots (union 28) on Los-loop. The Los-loop positive result therefore does not transfer to SZ-Taxi under the present construction.

Temporal-resolution variant of Los-loop.

Resampling Los-loop to 15-minute intervals (Sect. 4.9) preserves and strengthens the relative improvement of Mix over the graph-free baseline at matched wall-clock horizons of 15–60 minutes. At PH1 of the 15-minute series (15 minutes ahead), NoSpatial obtains 8.600 ± 0.249 , MultiGSL 7.246 ± 0.298 , and Mix 6.240 ± 0.187 (a 27.4% relative RMSE reduction versus NoSpatial; Mix wins 5/5 seeds). The relative reduction persists at PH2–PH4 of the variant (21.4%, 19.8%, 16.1%). Absolute RMSE magnitudes are not comparable across sampling intervals, and PH indices of the variant are not equivalent to PH indices of the 5-minute series. Full values are collected in Table 4.

Summary of the boundary.

Learned multi-lag structure with lag-aligned consumption helps where the learned graphs are informative (Los-loop at both sampling intervals). Where the learned graphs are nearly empty (SZ-Taxi), all learned variants collapse to the graph-free baseline.

Table 2: GCN family: test RMSE (mean, sample standard deviation in parentheses) over five seeds. GCN-MultiGSL consumes the union of the three multi-lag graphs as one static adjacency (Sect. 3.4).

Method	PH1	PH2	PH3	PH4
<i>Los-loop (5 min)</i>				
GCN (Physical)	8.14 (0.11)	8.54 (0.28)	8.76 (0.26)	8.76 (0.29)
GCN-NoSpatial	4.88 (0.31)	5.60 (0.39)	6.02 (0.28)	6.26 (0.26)
GCN-GSL	7.83 (0.11)	8.48 (0.21)	8.45 (0.19)	8.93 (0.12)
GCN-cGSL	5.76 (0.20)	6.33 (0.32)	6.68 (0.25)	6.88 (0.21)
GCN-MultiGSL	9.78 (0.14)	10.05 (0.16)	10.24 (0.13)	10.27 (0.11)
<i>SZ-Taxi (15 min)</i>				
GCN (Physical)	5.96 (0.01)	5.98 (0.00)	5.99 (0.00)	6.00 (0.00)
GCN-NoSpatial	4.11 (0.00)	4.15 (0.00)	4.19 (0.00)	4.22 (0.00)
GCN-GSL	4.88 (0.01)	4.91 (0.00)	4.93 (0.01)	4.96 (0.00)
GCN-cGSL	4.64 (0.00)	4.67 (0.00)	4.70 (0.00)	4.73 (0.00)
GCN-MultiGSL	4.82 (0.01)	4.85 (0.00)	4.88 (0.01)	4.90 (0.00)

Table 3: Matched-sparsity controls on Los-loop at PH1 only (Sect. 4.8). All graphs use a 30-edge budget under the canonical T-GCN protocol; five seeds except the capacity row, which is single-seed (seed 42) and labeled as such.

Configuration	RMSE	Edges / extra params	vs NoSpatial
RandTop30	6.10 (0.12)	30 random	+16.1%
CorrTop30	5.39 (0.10)	30 top- Pearson	+2.6%
T-GCN-NoSpatial	5.25 (0.19)	identity	—
T-GCN-MultiGSL	4.84 (0.11)	30 DAGMA lag slots	−7.8%
T-GCN-MultiGSL-Mix	4.49 (0.14)	same graphs, gated	−14.5%
NoSpatial $h=64$ (seed 42)	5.14	12,672 params	—
NoSpatial $h=74$ (seed 42)	5.14	16,872 params	$\approx 0\%$
Mix (seed 42)	4.46	17,091 params	−13.3%

6 Discussion

The results separate two design questions that are often conflated in graph-based traffic forecasting: *which graph is supplied to the backbone*, and *how that graph is used over the input window*. The discussion below reads the empirical pattern under that separation. We keep interpretation at the level of statistical dependency and architecture; we do not claim causal traffic mechanisms.

Table 4: Temporal-resolution variant of Los-loop at 15-minute sampling (Sect. 4.9). PH k denotes $15k$ minutes ahead and is *not* comparable to PH k of the 5-minute series. Five seeds.

Method	PH1 (15 min)	PH2 (30 min)	PH3 (45 min)	PH4 (60 min)
T-GCN-NoSpatial	8.600 (0.249)	9.311 (0.197)	10.048 (0.074)	10.460 (0.196)
T-GCN-MultiGSL	7.246 (0.298)	8.571 (0.327)	9.112 (0.077)	9.756 (0.210)
T-GCN-MultiGSL-Mix	6.240 (0.187)	7.322 (0.338)	8.063 (0.203)	8.777 (0.326)
Mix vs NoSpatial	−27.4%	−21.4%	−19.8%	−16.1%
Mix wins (of 5 seeds)	5	5	5	5

6.1 Why Dense Graphs Fail and What Learned Structure Adds

The physical road-network graph is a harmful default for these backbones at the operating points we study. On Los-loop the physical adjacency is dense (order 10^3 entries including self-loops for 207 sensors), yet the graph-free identity control reduces RMSE by roughly a quarter to a third relative to that graph at every horizon and on both datasets (Sects. 5.1 and 1). A simple reading is *oversmoothing* or, more generally, harmful aggregation: when almost every sensor is treated as a neighbor of every other, the graph convolution mixes signals that are not jointly informative for forecasting, and the recurrent or non-recurrent backbone cannot recover node-specific dynamics from the smeared representation. Identity adjacency freezes that mixing without removing the temporal path, which is consistent with the large, seed-stable gap between Physical and NoSpatial.

Learned single graphs (GSL, cGSL) occupy an intermediate position. They shrink the physical graph to a much smaller support (e.g. 28 contemporaneous edges on Los-loop) and usually improve on Physical, but they still lose to NoSpatial in every dataset×horizon cell (Sect. 5.2). Sparsity alone is not sufficient: at a matched 30-edge budget, random and top-correlation graphs are worse than no graph, while DAGMA-placed multi-lag edges with per-lag consumption are better (Sect. 5.3, Los-loop PH1 only). What the single learned graphs appear to add, relative to Physical, is a less harmful neighborhood; what they fail to add, relative to NoSpatial, is enough useful structure to pay for any residual mis-specification. The multi-lag experiments ask whether that residual can be recovered when dependency structure is allowed to be lag-specific and is consumed in a lag-aligned way.

This reading is consistent with the proximity-versus-dependency motivation: physical connectivity is an imperfect proxy for functional co-movement, and a dense proxy can be worse than no proxy. It does not imply that graph-based forecasting is generally harmful; it implies that the default reference graph should be tested against a graph-free control before learned structure is credited with gains.

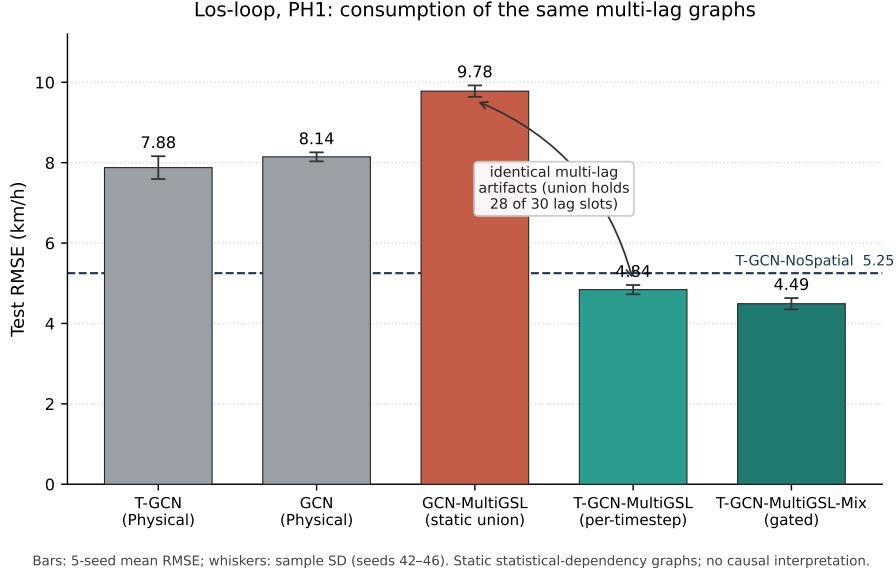


Fig. 1: Los-loop PH1: the same multi-lag DAGMA artifacts under static-union consumption (GCN-MultiGSL) versus per-timestep consumption (T-GCN-MultiGSL), with physical and gated references. Bars are five-seed mean RMSE; whiskers are sample standard deviations (seeds 42–46). The union retains 28 of the 30 lagged edge slots. Graphs are static statistical-dependency estimates; the figure does not encode causal influence.

6.2 Structure Versus Consumption

The central architectural observation is the dissociation in Sect. 5.4. The three lag-specific DAGMA graphs are fitted once and shared. Consumed as a static union by GCN-MultiGSL, they yield 9.78 ± 0.14 RMSE on Los-loop PH1; consumed per timestep by T-GCN-MultiGSL, the same artifacts yield 4.84 ± 0.11 . The union retains 28 of the 30 lagged edge slots, so the bulk of the edge content is present in both conditions. The opposite outcomes therefore *cannot be explained by the edge set alone*. Because GCN and T-GCN also differ in backbone (non-recurrent versus recurrent), this comparison is not a fully controlled isolation of consumption; it provides evidence that graph consumption is a major determinant of whether the learned multi-lag structure is useful.

Within the T-GCN family, the consumption ladder is equally informative. Fixed cyclic lag assignment already beats NoSpatial on Los-loop (7.8% relative RMSE reduction at PH1). Adding three global scalars (Weighted) changes almost nothing (differences ≤ 0.01 RMSE), so a single static mixture over lags is not what drives the gain. Replacing that global mixture with a per-node, per-timestep gate (Mix) adds a further 0.35–0.40 RMSE on Los-loop in 5/5 seeds at every horizon (Sect. 5.5), without modifying the learned edge sets. The results thus indicate that the utility of the

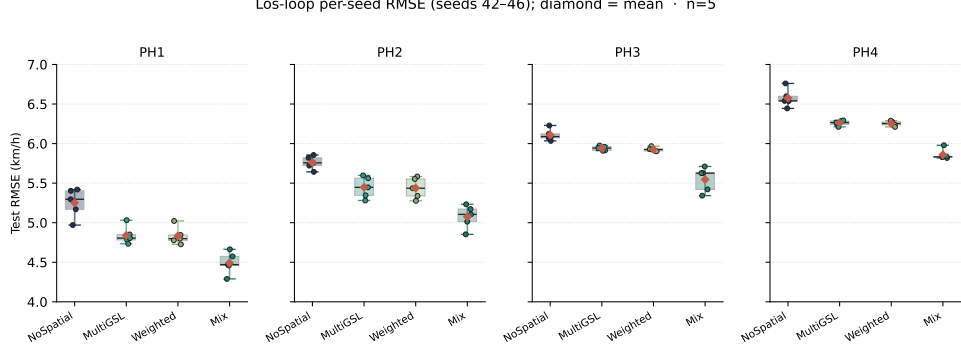


Fig. 2: Los-loop per-seed test RMSE for the multi-lag consumption ladder (seeds 42–46). Diamonds mark five-seed means matching Table 1. $n = 5$; see Sect. 4.7.

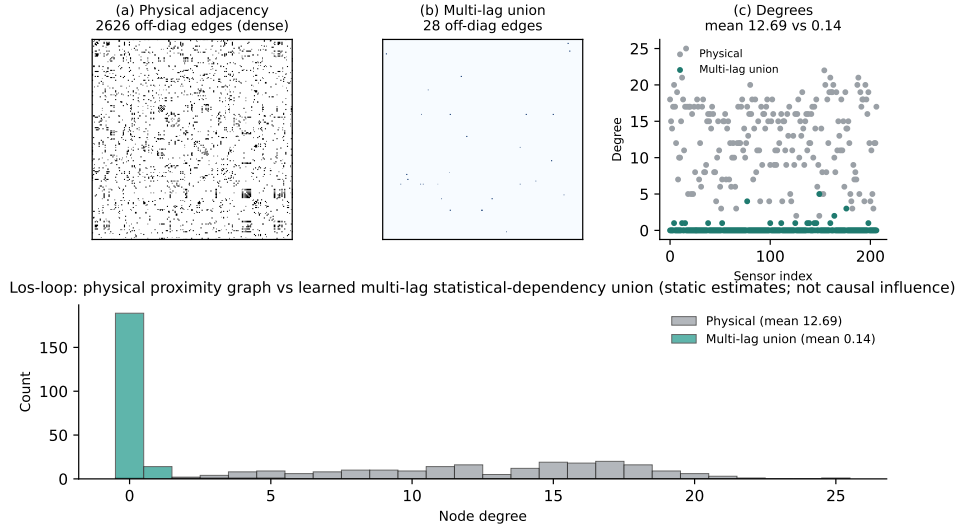


Fig. 3: Los-loop graph structure: (a) physical road-network adjacency (2626 off-diagonal edges; mean degree 12.69); (b) union of the three multi-lag DAGMA graphs after consumer thresholding (28 edges; mean degree 0.14); (c) and histogram of node degrees. Learned graphs describe statistical dependency structure, not causal traffic influence.

learned multi-lag structure depends critically on how the graphs are consumed: lag-aligned use helps where the graphs are informative, and finer selection helps further. The gate varies *use*, not *structure*; the graphs remain static fitted artifacts.

The cGSL–GSL contrast is a miniature of the same theme. Binary symmetrization of the same contemporaneous artifact helps the non-recurrent GCN substantially (cGSL 5.76 vs GSL 7.83 on Los-loop PH1) and is nearly irrelevant for T-GCN (5.82 vs

5.86). One consistent explanation is *aggregation compatibility*: a symmetric aggregator is better matched to a symmetric operator, whereas the recurrent cell can absorb directed asymmetry without the same penalty. We offer this as an observation of the current study, not as a general theorem about DAG versus cyclic graphs, and we do not revive a temporal reading of the contemporaneous DAG.

6.3 What the Learned Graph Represents

We describe the learned graphs as descriptive statistical dependency structure under a linear DAGMA fit with an acyclicity regularizer. The multi-lag construction stacks explicit lag blocks, and the consumption ablations show that treating those blocks as lag-specific operators is more effective than merging them into one static adjacency. That pattern is *consistent with* a lag-structured reading of the fitted matrices, but we did not independently validate the lag interpretation against ground-truth propagation, and acyclicity is an optimization constraint rather than causal evidence.

The dataset boundary sharpens this reading. On SZ-Taxi the multi-lag fit retains only two lagged edges, and every learned variant remains within seed variability of the graph-free control (Sect. 5.6). On Los-loop, where the lag blocks are substantially denser (12/3/15 slots; union 28), lag-aligned consumption yields the study’s positive result, and the 15-minute resolution variant preserves a large relative improvement over NoSpatial at 15–60 minutes ahead (Table 4). The honest synthesis is conditional: learned multi-lag graphs help forecasting when the fitted dependency structure is informative enough to be worth consuming, and when the consumption mechanism respects the lag organization of that structure. Where the fit is nearly empty, the method collapses to the graph-free baseline.

None of this implies that the physical graph is useless in absolute terms, or that learned graphs will help every network. It implies that the default practice of treating a dense proximity graph as ground truth, and of crediting any learned graph that improves on it, is incomplete without a graph-free control and without an explicit account of how the graph is consumed over time.

7 Limitations

The study is a controlled evaluation on two public benchmarks, not a claim of universal benefit. Several limitations bound the evidence.

Statistical power.

All main results aggregate five training seeds. Sample standard deviations and paired win counts are reported, and paired t -tests are given for reference, but with $n = 5$ the exact Wilcoxon signed-rank test cannot fall below $p = 0.0625$. We therefore avoid definitive significance claims and rely on effect size, seed consistency, and the matched controls.

Scope of the controls.

The matched-sparsity and capacity controls are restricted to Los-loop at PH1. There is no matched-sparsity control on SZ-Taxi or at longer horizons, no sparsified variant of

the physical graph, and no sensitivity sweep over DAGMA λ_1 or threshold parameters. The sparsity conclusion is explicitly scoped to that single cell.

Datasets and horizons.

Only two datasets are used, with different sampling intervals (5 min and 15 min). Horizons are limited to $\text{PH} \leq 4$ at the native sampling rate of each series. The 15-minute Los-loop experiment covers 15–60 minutes ahead as a temporal-resolution variant; it is not evidence for PH5–8 at 5-minute sampling, and PH indices are not comparable across sampling intervals.

Graph-learning assumptions.

Graphs are estimated with linear DAGMA (ℓ_2 score, acyclicity regularizer). Nonlinear dependency, measurement noise beyond the linear SEM assumption, and misspecification of the lag depth $L = 3$ are not examined. Measured runtimes are non-negligible for the multi-lag fit (on the order of hours for the 828-variable construction on Los-loop), which limits rapid replication at larger N .

Static graphs; gated use.

The learned graphs are fitted once and remain static. The Mix mechanism varies how those static graphs are consumed per node and timestep; it does not produce time-varying edge sets. Genuinely time-varying learned graphs remain future work.

Backbone scope.

Experiments cover the GCN and T-GCN family used here. Other spatio-temporal architectures (attention-based models, diffusion variants, large transformers) are outside the study.

8 Conclusion

We asked when and how learned graph structure helps graph-based traffic forecasting, separating the *source* of the graph from the *mechanism that consumes it*. Across two public benchmarks, twelve configurations, four horizons, and five seeds, three conclusions follow. First, the dense physical road-network graph is not a sound default: a graph-free identity control outperforms it in every dataset–horizon cell, and single learned contemporaneous graphs improve on the physical graph but never on the graph-free control. Second, lag-specific multi-lag DAGMA graphs are useful on Los-loop only when consumed in a temporally aligned manner; feeding the identical graphs as a static union is markedly worse, so consumption is at least as important as learning, while a global lag mixture is insufficient and a per-node, per-timestep gate adds a further seed-consistent gain (up to 14.5% relative RMSE reduction versus the graph-free baseline at PH1). Third, the benefit is conditional: on SZ-Taxi, where the learned multi-lag graphs retain almost no edges, all learned variants remain within seed variability of the graph-free baseline, and matched-budget controls show that sparsity alone does not explain the Los-loop gain.

Future work includes longer horizons at 5-minute resolution, additional networks, sparsified-physical controls, λ /threshold sensitivity, other backbones, and genuinely time-varying graph structure. Practically, we recommend that graph-based traffic forecasters report a graph-free control and state explicitly how any learned graph is consumed over time.

References

- [1] Akhtar, M., Moridpour, S.: A review of traffic congestion prediction using artificial intelligence. *Journal of Advanced Transportation* **2021** (2021)
- [2] Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. In: *ICLR* (2017)
- [3] Zhao, L., Song, Y., Zhang, C., Liu, Y., Wang, P., Lin, T., Deng, M., Li, H.: T-GCN: A temporal graph convolutional network for traffic prediction. *IEEE Trans. on Intell. Transportation Systems* **21**(9), 3848–3858 (2020)
- [4] Chen, Y., Wu, L.: In: Wu, L., Cui, P., Pei, J., Zhao, L. (eds.) *Graph Neural Networks: Graph Structure Learning*, pp. 297–321. Springer, Singapore (2022). https://doi.org/10.1007/978-981-16-6054-2_14 . https://doi.org/10.1007/978-981-16-6054-2_14
- [5] Zheng, X., Aragam, B., Ravikumar, P., Xing, E.P.: Dags with no tears: Continuous optimization for structure learning. In: *NeurIPS* (2018)
- [6] Bello, K., Aragam, B., Ravikumar, P.: Dagma: learning dags via m-matrices and a log-determinant acyclicity characterization. In: *Proceedings of the 36th International Conference on Neural Information Processing Systems. NIPS '22*. Curran Associates Inc., Red Hook, NY, USA (2024)
- [7] Hamed, M.M., Al-Masaeid, H.R., Said, Z.M.B.: Short-term prediction of traffic volume in urban arterials. *Journal of Transportation Engineering* **121**(3), 249–254 (1995)
- [8] Smola, A.J., Schölkopf, B.: A tutorial on support vector regression. *Statistics and Computing* **14**(3), 199–222 (2004)
- [9] Lippi, M., Bertini, M., Frasconi, P.: Short-term traffic flow forecasting. *IEEE Transactions on Intelligent Transportation Systems* **14**(2), 871–882 (2013)
- [10] Zhang, J., Zheng, Y., Qi, D.: Deep spatio-temporal residual networks for citywide crowd flows prediction, 1655–1661 (2017)
- [11] Cao, X., Zhong, Y., Zhou, Y., Wang, J., Zhu, C., Zhang, W.: Interactive temporal recurrent convolution network for traffic prediction. *IEEE Access* **5**, 9893–9901 (2017)

- [12] Bai, J., Zhu, J., Song, Y., Zhao, L., Hou, Z., Du, R., Li, H.: A3t-gcn: Attention temporal graph convolutional network for traffic forecasting. *ISPRS International Journal of Geo-Information* **10**(7) (2021) <https://doi.org/10.3390/ijgi10070485>
- [13] Sun, C., Li, C., Lin, X., Zheng, T., Meng, F., Rui, X., Wang, Z.: Attention-based graph neural networks: a survey. *Artificial Intelligence Review* **56**(2), 2263–2310 (2023) <https://doi.org/10.1007/s10462-023-10577-2>